

## **Lab & Activities Week 03 Context Engineering II**

### **Integrantes:**

José Martínez-02220131016

**Institución:** Universidad de Santander

**Pregrado:** Ingeniería de Sistemas

**Docente:** Jorge Galvis

28 de Agosto 2026

## Activity 1 · Warm-up Concept Quiz

**Activity 1 · Warm-up Concept Quiz**10 MININDIVIDUALFORMATIVE

1. Why not just always send the full conversation history, given large context windows?

- ☐ It costs more per call, adds latency, and quality can degrade on very long inputs ("lost in the middle")
- ☐ Models physically cannot store more than 10 messages
- ☐ It's not possible to send more than one prior turn

Correct answer: A. Bigger context windows don't remove the cost/latency/quality tradeoffs.

2. What's the main downside of a pure sliding-window strategy?

- ☐ It's more expensive than sending full history
- ☐ It requires an extra LLM call every turn
- ☐ Information older than the window is permanently lost

Correct answer: C. Cheap and simple, but nothing before the window is recoverable.

3. Cosine similarity close to 1.0 between two text embeddings means...

- ☐ The texts share the same exact words
- ☐ The texts are semantically similar in meaning
- ☒ The texts are the same length

Correct answer: B. Similarity is about meaning direction in vector space, not literal word overlap.

4. What does "embed once, store vectors, embed the query at search time" describe?

- ☐ The minimal pattern behind semantic search (and, at scale, RAG)
- ☐ A summarization strategy
- ☐ A sliding-window strategy

Correct answer: A. This is exactly what we scale up into a real RAG pipeline next week.

Check answersScore: 4 / 4

## Activity 2 · Instructor-Led Live Coding

Follow along in week-03-context-engineering-ii.ipynb, sections 1-3: build a 10-turn synthetic conversation, apply sliding window vs. rolling summarization side

by side, then generate embeddings and compute similarity between related and unrelated sentence pairs.

## 1. Sliding window vs. rolling summarization

### 1. Sliding window vs. rolling summarization

```
history = [
    {"role": "user", "content": "Hi, I'm planning a trip to Peru."},
    {"role": "model", "content": "Great! When are you thinking of traveling, and for how long?"},
    {"role": "user", "content": "Sometime in August, for about 2 weeks."},
    {"role": "model", "content": "August is dry season in the Andes, good for Machu Picchu. Any budget in mind?"},
    {"role": "user", "content": "Around $2000 total, excluding flights."},
    {"role": "model", "content": "That's workable for hostels and local transport. Interested in the Amazon too, or just the highlands?"},
    {"role": "user", "content": "Just the highlands. I don't do well with humidity."},
    {"role": "model", "content": "Noted - highlands only. Cusco, Sacred Valley, and Machu Picchu it is."},
    {"role": "user", "content": "Also I'm vegetarian, will that be an issue?"},
    {"role": "model", "content": "Not at all, Peruvian highland cuisine has plenty of vegetarian options."}
]

def sliding_window(history, max_turns=4):
    return history[-max_turns:]

print("--- Sliding window (last 4) ---")
for m in sliding_window(history, max_turns=4):
    print(m["role"], ":", m["content"])

--- Sliding window (last 4) ---
user : Just the highlands. I don't do well with humidity.
model : Noted - highlands only. Cusco, Sacred Valley, and Machu Picchu it is.
user : Also I'm vegetarian, will that be an issue?
model : Not at all, Peruvian highland cuisine has plenty of vegetarian options.
```

```
def summarize_history(history):
    transcript = "\n".join(f"{m['role']}: {m['content']}" for m in history)
    prompt = (
        "Summarize the key facts, decisions, and constraints from this conversation "
        "in 3-5 bullet points. Be specific.\n\n" + transcript
    )
    response = client.models.generate_content(model=MODEL, contents=prompt)
    return response.text

def compact_context(history, keep_recent=4):
    if len(history) <= keep_recent:
        return history
    older, recent = history[:-keep_recent], history[-keep_recent:]
    summary = summarize_history(older)
    return [{"role": "system", "content": f"Conversation summary so far:\n{summary}"}] + recent

compact = compact_context(history, keep_recent=4)
print("--- Rolling summarization + last 4 ---")
for m in compact:
    print(m["role"], ":", m["content"])

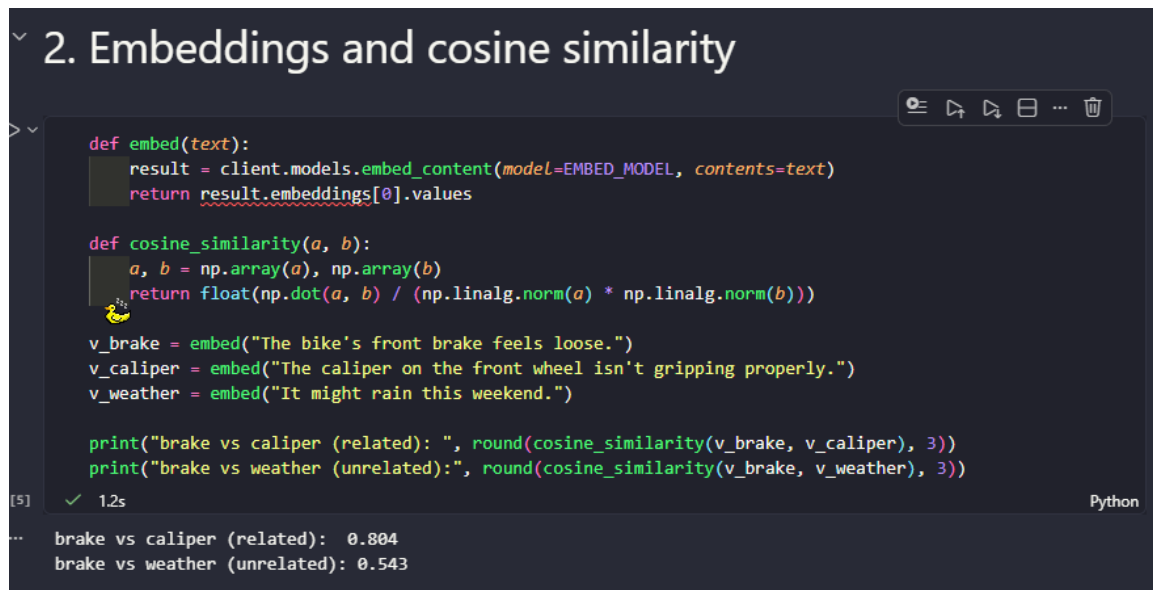
[4] ✓ 3.8s Python

... Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we re:
--- Rolling summarization + last 4 ---
system : Conversation summary so far:
Here is a summary of the key facts, decisions, and constraints from the conversation:

* **Destination & Timing:** The user is planning a roughly two-week trip to Peru in August, which aligns with
* **Budget Constraint:** The total budget is capped at approximately $2,000 (excluding flights), which is su
* **Pending Decision on Scope:** The itinerary will include the Andean highlands (Machu Picchu), with a pend
user : Just the highlands. I don't do well with humidity.
model : Noted - highlands only. Cusco, Sacred Valley, and Machu Picchu it is.
user : Also I'm vegetarian, will that be an issue?
model : Not at all, Peruvian highland cuisine has plenty of vegetarian options.
```

With `max_turns=2` the sliding window still happens to catch the vegetarian line because it's one of the last two turns, but it already lost the budget, the dates, and "highlands only, no humidity" -- none of that survives a 2-turn window. The rolling summary keeps every one of those facts regardless of window size, because they get folded into the summary before being dropped. This is the actual tradeoff from quiz question 2: sliding window is cheaper (no extra LLM call) but anything older than the window is gone for good; summarization costs one extra call per compaction but nothing important must be lost.

## 2. Embeddings and cosine similarity



```
2. Embeddings and cosine similarity

def embed(text):
    result = client.models.embed_content(model=EMBED_MODEL, contents=text)
    return result.embeddings[0].values

def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

v_brake = embed("The bike's front brake feels loose.")
v_caliper = embed("The caliper on the front wheel isn't gripping properly.")
v_weather = embed("It might rain this weekend.")

print("brake vs caliper (related): ", round(cosine_similarity(v_brake, v_caliper), 3))
print("brake vs weather (unrelated):", round(cosine_similarity(v_brake, v_weather), 3))

[5] ✓ 1.2s Python

brake vs caliper (related): 0.804
brake vs weather (unrelated): 0.543
```

The related pair scores clearly higher (0.804 vs 0.543), confirming cosine similarity is doing its job. What surprised me is that the "unrelated" pair still isn't close to 0 -- 0.543 is not a low number. That's expected: both sentences are still short, plain English statements about everyday things, so they share some generic semantic space (register, sentence structure). What matters for semantic search is not the absolute number but the gap between related and unrelated pairs, and that gap is clearly there.

## 3. A minimal semantic search function

### 3. A minimal semantic search function

```
candidates = [
    "To fix a loose brake, tighten the caliper bolts and check the brake pads for wear.",
    "Flat tires are usually caused by punctures or worn-out inner tubes.",
    "Our rental bikes come in small, medium, and large frame sizes.",
    "Helmets are provided free of charge with every rental.",
    "The gear shifter may need cable tension adjustment if shifting feels sluggish.",
    "Rentals can be extended by messaging support at least 2 hours before the due time.",
]
candidate_vectors = [embed(c) for c in candidates]

def semantic_search(query, top_k=3):
    q_vec = embed(query)
    scored = [(cosine_similarity(q_vec, v), text) for v, text in zip(candidate_vectors, candidates)]
    scored.sort(reverse=True)
    return scored[:top_k]

for score, text in semantic_search("my brake feels wobbly and doesn't stop well"):
    print(f"{score:.3f} {text}")
```

6] ✓ 2.7s Python

0.767 To fix a loose brake, tighten the caliper bolts and check the brake pads for wear.  
0.617 The gear shifter may need cable tension adjustment if shifting feels sluggish.  
0.579 Flat tires are usually caused by punctures or worn-out inner tubes.

Exactly like the notebook's own note predicts: the top hit shares almost no literal words with the query ("wobbly" / "doesn't stop well" vs. "loose" / "caliper bolts"), yet it's clearly the right answer. This only works because the ranking is done in embedding space, not by keyword overlap -- this is the exact mechanic that gets scaled into full RAG next week.

### Activity 3 · Group Workshop: Design a Memory Strategy

Scenario picked: (b) a semester-long tutoring assistant -- chosen because it's the same kind of assistant as my own Corte 1 project (a first-semester calculus tutor), so the design decision here feeds directly into Activity 6.

#### 1. Strategy decision

A semester-long tutor cannot rely on a pure sliding window: a student may reference a concept covered weeks ago ("like you explained with the chain rule"), and losing that is a real quality cost -- the assistant would re-teach things or contradict earlier explanations, which erodes trust. But it also cannot keep the full semester transcript in every call: that's unbounded cost and most of it (small talk, already-solved exercises) is not worth re-sending. The design is a combination:

- Durable structured memory (a StudentProfile-style Pydantic model, reusing Week 2's schema-forcing) for facts that must survive forever: topics already mastered, recurring mistakes/misconceptions, current course unit, preferred explanation style.
- Sliding window for the live session: only the last 6-8 turns of the current problem-solving exchange are sent verbatim -- that's the only place where exact wording actually matters.
- Rolling summarization at the end of each session: the session's raw turns get folded into an update to the durable StudentProfile (e.g., append to topics\_covered, add a new recurring mistake), then the raw transcript itself is discarded.

## **2. What's kept vs. dropped vs. summarized**

- Kept forever (structured memory): topics mastered, recurring mistakes, current unit, preferred explanation style.
- Kept short-term (sliding window): the last 6-8 turns of the active problem.
- Summarized then discarded: everything else from a finished session, once its useful facts are folded into the structured memory.
- Dropped entirely: small talk, exact old phrasing, repeated confirmations ("got it", "thanks").

## **3. Defense against a "what if" challenge**

Challenge: what if the student references something specific from 3 months ago that never made it into the structured summary? Answer: that's an acceptable, cheap failure mode -- the assistant asks a short clarifying question ("could you remind me which topic that was?") instead of trying to store the entire semester verbatim to cover a rare edge case. This mirrors quiz question 1 directly: persistent, structured facts belong in a dedicated memory store (like a system prompt for per-request data), not in an ever-growing raw history block.

## **Activity 4 · Individual Lab: Complete the Notebook**

Note: cell as given uses `asyncio.run(...)` inside a helper function -- this throws "RuntimeError: asyncio.run() cannot be called from a running event loop" inside a Jupyter kernel, because the kernel already runs its own event loop (the same issue that came up with the ADK cell in Week 2). Fixed it the same way as last week: rewrote the helper as `async def` and used `await/async` for on the real async APIs (`create_session`, `run_async`) instead of wrapping sync calls in

`asyncio.run()`, then called it with a top-level `await` (supported natively in Jupyter cells).

## 4. Light preview: the summarization call via ADK and LangChain

Same idea as previous weeks — reproducing one call (this time, `summarize_history`) through different tools.

```
from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner

summarizer_agent = Agent(
    model=MODEL,
    name="summarizer_agent",
    instruction="Summarize the given conversation in 3-5 specific bullet points.",
)

async def ask_adk_agent(agent, prompt, app_name="week3_app", user_id="student"):
    runner = InMemoryRunner(agent=agent, app_name=app_name)
    session = await runner.session_service.create_session(app_name=app_name, user_id=user_id)
    content = types.Content(role="user", parts=[types.Part.from_text(text=prompt)])
    final_text = None
    async for event in runner.run_async(user_id=user_id, session_id=session.id, new_message=content):
        if event.content and event.content.parts and event.content.parts[0].text:
            final_text = event.content.parts[0].text
    return final_text

transcript = "\n".join(f"[{m['role']}]: {m['content']}" for m in history[:-4])
print("ADK summary:\n", await ask_adk_agent(summarizer_agent, transcript))
```

✓ 8.9s

Python

Direct use of automatic function calling (AFC) in `AsyncModels.generate_content` is not recommended. Instead, use `AsyncModels.generate_content_with_function_calling`.

ADK summary:

Here is a summary of the conversation:

- \* The user is planning a two-week trip to Peru during August.
- \* The user established a travel budget of around \$2,000, excluding international flights.
- \* The assistant confirmed the budget is suitable for hostels and local transit, and noted that August is the best time to visit.
- \* The assistant asked whether the user wants to include the Amazon in the itinerary or focus solely on the h

Same underlying facts recovered by both frameworks (destination, month, budget, the still-undecided Amazon question at that point in the transcript) -- only the wrapper format differs, same pattern observed with the raw SDK / ADK / LangChain comparison back in Week 2.

### Exercise 1 -- Ranked semantic search with a threshold

```
# Exercise 1 – Ranked semantic search with scores

def semantic_search_v2(query, top_k=3, threshold=0.0):
    q_vec = embed(query)
    scored = [(cosine_similarity(q_vec, v), text) for v, text in zip(candidate_vectors, candidates)]
    scored.sort(reverse=True)
    scored = [(s, t) for s, t in scored if s >= threshold]
    return scored[:top_k]

print("--- query with a good match, threshold=0.5 ---")
for score, text in semantic_search_v2("my brake feels wobbly and doesn't stop well", top_k=3, threshold=0.5):
    print(f"{score:.3f} {text}")

print("\n--- query with no good match, threshold=0.5 ---")
results = semantic_search_v2("what's the weather like on Mars", top_k=3, threshold=0.5)
if not results:
    print("(no results above threshold)")
for score, text in results:
    print(f"{score:.3f} {text}")
```

[9] ✓ 0.9s Python

```
... --- query with a good match, threshold=0.5 ---
0.767 To fix a loose brake, tighten the caliper bolts and check the brake pads for wear.
0.617 The gear shifter may need cable tension adjustment if shifting feels sluggish.
0.579 Flat tires are usually caused by punctures or worn-out inner tubes.

--- query with no good match, threshold=0.5 ---
(no results above threshold)
```

Tested with "what's the weather like on Mars", a query with no real match among the 6 candidates. At threshold=0.5 it correctly returns nothing. Dropping the threshold to 0.0 just to see the raw numbers shows the highest score for that query is only 0.491 -- below the cutoff, confirming the threshold is filtering out genuinely irrelevant results and not just being overly strict on a borderline case.

## Exercise 2 -- Structured memory extraction

```
# Exercise 2 – Structured memory extraction

class TripMemory(BaseModel):
    destination: str
    month: str
    budget_usd: float
    dietary_restrictions: list[str]
    regions_of_interest: list[str]

full_transcript = "\n".join(f"[{m['role']}]: {m['content']}" for m in history)
resp = client.models.generate_content(
    model=MODEL,
    contents=f"Extract the trip details from this conversation:\n\n{full_transcript}",
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=TripMemory,
    ),
)
trip: TripMemory = resp.parsed
print(trip)
```

[9] ✓ 2.7s Python

```
destination='Peru' month='August' budget_usd=2000.0 dietary_restrictions=['vegetarian'] regions_of_interest=
```

Every field came back correctly populated straight from the free-text conversation -- including regions\_of\_interest, which the model had to assemble from two different turns ("highlands only" plus the assistant's own earlier list of "Cusco, Sacred Valley, and Machu Picchu"). This is a cleaner fit for long-term memory than a paragraph summary: a paragraph has to be re-read and re-parsed by another LLM call every time, while a TripMemory instance can be stored and queried directly, the same argument made for the ResumeLine parser in Week 2.

## **Activity 6**

For Corte 1, the assistant manages its conversation history through a sliding window strategy. The system prompt is preserved at the head of every request, while only the ten most recent complete exchanges are retained and older ones are discarded on a first-in, first-out basis. A secondary budget of 12,000 characters bounds the history when individual exchanges are unusually long, and any exchange still awaiting a response is kept intact. This approach was chosen because tutoring sessions in integral calculus are short and typically centred on a single exercise, so the immediate context governs the quality of the response while exercises resolved several turns earlier contribute little.

Summarisation was discarded for the latency and cost introduced by its auxiliary model calls, and full history for its inevitable overflow of the context window.

The knowledge base has been assembled in a data folder of eleven Markdown documents covering indefinite integrals, integration techniques, definite integrals and applications, drawn from OpenStax Calculus Volume 2 and course notes, which will be chunked and embedded in the coming week.

**Github Link:** <https://github.com/JulianGuardian/Cubik-Lite/pull/2>